

# Token Ring Banking System

A distributed banking system implementation using the Token Ring mutual exclusion algorithm. This project simulates a distributed ATM network where 4 independent nodes safely access and modify a shared bank account balance.

## Overview

This system demonstrates distributed mutual exclusion through a Token Ring architecture, where 4 independent ATM nodes coordinate access to a shared resource (bank account) by passing a token around a logical ring.

## Documentation and Demo

### Video Demonstration

Watch the system in action: [YouTube Demo Video](#)

The video demonstrates:

- Normal token circulation through all 4 nodes
- Transaction execution with mutual exclusion
- Coordinator election using Bully Algorithm
- Token regeneration after node failure (Scenario A)
- Ring reconstruction without failed nodes (Scenario B)
- Node recovery and reintegration (Scenario C)

### Technical Documentation

Complete project documentation: [PDF Documentation](#)

The PDF includes:

- Detailed system architecture and design decisions
- Token Ring and Bully Algorithm implementation details
- Fault tolerance mechanisms and consensus voting
- Complete testing scenarios and results
- Code documentation and API reference

### Key Features

- **Token Ring Algorithm:** Only the node holding the token can execute transactions
- **Bully Algorithm:** Distributed coordinator election for fault recovery
- **Distributed Architecture:** 4 independent nodes running on localhost (ports 3001-3004)
- **Safe Transactions:** Atomic deposit and withdrawal operations with mutual exclusion
- **Message-based Coordination:** No shared memory, all communication via HTTP message passing
- **Transaction Queueing:** Each node can queue multiple transactions, executed when token arrives
- **Advanced Fault Tolerance:**
  - Automatic retry for temporary failures
  - Token regeneration when lost (Scenario A)
  - Dynamic ring reconstruction without failed nodes (Scenario B)
  - Coordinator election using Bully Algorithm
  - Node recovery and reintegration
- **Complete Logging:** Full audit trail of token circulation, elections, and transaction execution

## System Architecture

The system consists of 4 ATM nodes arranged in a logical ring:

ATM1 → ATM2 → ATM3 → ATM4 → ATM1

Each node:

- Runs as a separate process on localhost
- Knows only its successor in the ring
- Can perform deposits and withdrawals
- Passes the token when not in use
- Can participate in coordinator elections
- Can become the coordinator if elected

### Coordinator Role

The system includes a **coordinator node** elected through the Bully Algorithm:

- **Responsibilities:**
  - Regenerate lost tokens (Scenario A)
  - Reconstruct ring topology when nodes fail (Scenario B)

- Handle node recovery announcements
- Reintegrate recovered nodes into the ring
- **Election:**
  - Initially, the highest ID node (ATM4) is coordinator
  - When coordinator fails, any node can trigger election
  - Highest ID active node always becomes coordinator
- **Not a central server:** The coordinator is just a regular node with additional responsibilities, maintaining the distributed nature of the system

## How It Works

---

### Token Circulation

1. **Single Token:** A unique token circulates continuously through the ring, carrying the current account balance
2. **Mutual Exclusion:** Only the token holder can access the critical section (execute transactions)
3. **Token Forwarding Logic** (according to Token Ring algorithm):
  - **Has pending transactions** → Executes ONE transaction, then forwards token to successor
  - **No pending transactions** → Forwards token IMMEDIATELY to successor
4. **Display Delay** (educational only): A configurable `TOKEN_DISPLAY_DELAY` (default 5 seconds) is added for visualization purposes when no transactions are pending. This is NOT part of the original algorithm and should be set to 0 in production implementations.

### Transaction Execution

When a node receives the token:

1. **Check pending transactions:** If transactions are queued, execute ONE transaction
2. **Execute atomically:**
  - Read current balance from token
  - Validate operation (sufficient funds for withdrawal)
  - Calculate new balance
  - Update balance in token
  - Log transaction with timestamp
3. **Forward token:** Pass token with updated balance to next node
4. **Repeat:** If more transactions are pending, execute them when token returns

### Shared Resource Management

The system implements a **distributed shared resource** (bank account balance) without shared memory:

- The balance travels **inside the token** as it circulates
- Each node operates on the balance only when holding the token
- Token passing via HTTP POST ensures **message-based coordination**
- No global variables or shared memory between nodes
- This respects the "no shared memory" constraint of distributed systems

### Fault Tolerance & Coordinator Election

The system implements comprehensive fault tolerance through the **Bully Algorithm** for coordinator election and automatic recovery mechanisms.

#### Bully Algorithm for Coordinator Election

When a node detects coordinator failure, it initiates an election:

1. **Election Process:**
  - Node sends ELECTION messages to all nodes with higher IDs
  - If a higher node responds with OK, it takes over the election
  - If no OK response within timeout, the node declares itself coordinator
  - New coordinator sends COORDINATOR announcement to all lower nodes
2. **Initial State:**
  - At startup, the highest ID node (ATM4) is assumed to be the initial coordinator
  - If the coordinator fails, any node can start an election
3. **Properties:**
  - **Safety:** Only one coordinator at a time
  - **Liveness:** System eventually elects a coordinator
  - **Optimality:** The highest ID active node becomes coordinator

### Failure Scenarios and Recovery

#### Scenario A: Token Lost (Node with Token Crashes)

The system implements automatic token lost detection with **consensus-based decision making** to prevent duplicate tokens:

1. **Token Circulation Monitoring:**
  - Each node tracks `lastTokenSeen` timestamp (updated on receive/forward)

- Every 5 seconds, nodes check if token circulation timeout (30s) is exceeded
- If timeout exceeded → token is declared lost

## 2. Detection and Coordinator Election:

- Node detecting loss logs: `*** TOKEN LOST DETECTED ***`
- Election is triggered to select a coordinator for handling the situation
- All nodes participate in Bully Algorithm election

## 3. Consensus Vote (Critical for Preventing Duplicates):

- **Coordinator conducts a token status vote** among all active nodes
- Each node reports: `hasToken: boolean, tokenId, and balance`
- Coordinator collects all responses and makes decision:
  - **If ANY node has the token** → Scenario B (no regeneration needed)
  - **If NO node has the token** → True token loss, proceed to regeneration

## 4. Token Invalidation (Before Regeneration):

- Coordinator broadcasts `INVALIDATE_TOKEN` command to all nodes
- All nodes discard any old tokens they may have
- This prevents race conditions where a node might have an old token

## 5. Token Regeneration:

- Coordinator regenerates token using highest balance from consensus votes
- Token gets new UUID but preserves account balance
- New token ID is broadcast in topology update
- All nodes validate incoming tokens against the new token ID

## 6. Circulation Resumes:

- Coordinator starts circulating the regenerated token
- All nodes restart their circulation monitoring
- System continues normal operation with preserved account balance

### Why Consensus Prevents Duplicates:

- Without consensus, multiple nodes detecting "token lost" simultaneously could each start elections and regenerate tokens independently
- The coordinator's consensus vote ensures only ONE decision is made by ONE coordinator
- Token invalidation guarantees no old tokens survive the regeneration process
- Token ID validation prevents duplicate tokens from being accepted

### Important: ATM1 special behavior to prevent duplicate tokens:

- On startup, ATM1 checks if other nodes are active
- If other nodes found → treats as recovery (no token creation)
- If no other nodes → creates initial token (first system startup)
- This prevents creating duplicate tokens when ATM1 recovers

### Scenario B: Node Failure Without Token

1. Node detects successor is unreachable (automatic retries fail)
2. Election is triggered to select a coordinator
3. Coordinator reconstructs ring topology without failed node
4. New topology is broadcast to all active nodes
5. Token circulation continues with updated ring structure

### Scenario C: Node Recovery

1. Recovered node announces its return to the coordinator
2. Coordinator verifies node health
3. Coordinator reintegrates node into ring topology
4. Updated topology is broadcast to all nodes
5. Normal circulation resumes with recovered node included

### Fault Tolerance Features

- **Token circulation monitoring:** Each node monitors token circulation with 30-second timeout
- **Automatic retry mechanism:** Handles temporary unavailability (5 retries with 2-second delays)
- **Token regeneration:** Coordinator recreates lost tokens with preserved balance
- **Balance preservation:** Each node tracks last known balance for accurate token regeneration
- **Dynamic topology:** Ring automatically adjusts to node failures/recoveries
- **Coordinator election:** Bully Algorithm ensures a coordinator is always available
- **Node health checks:** Coordinator validates recovered nodes before reintegration
- **Duplicate token prevention:** ATM1 checks for active nodes before creating initial token
- **No single point of failure:** Any active node can become coordinator

## Description

---

Built with [NestJS](#) framework.

## Technical Requirements

---

- Node.js (v18 or higher)
- npm

- 4 available ports on localhost (default: 3001-3004)
- PowerShell (for Windows automation scripts)

## Quick Start

---

### Installation

```
npm install
```

### Starting the System

#### Option 1: Automated (recommended)

```
# Start all 4 nodes in separate PowerShell windows
.\start-nodes.ps1
```

#### Option 2: Manual (for debugging)

```
# Terminal 1 - ATM1
$env:NODE_ID=1; $env:PORT=3001; npm run start:dev

# Terminal 2 - ATM2
$env:NODE_ID=2; $env:PORT=3002; npm run start:dev

# Terminal 3 - ATM3
$env:NODE_ID=3; $env:PORT=3003; npm run start:dev

# Terminal 4 - ATM4
$env:NODE_ID=4; $env:PORT=3004; npm run start:dev
```

### Basic Operations

#### Send Transactions:

```
# Deposit $200 to ATM2
.\transaction.ps1 -NodeId 2 -Type deposit -Amount 200

# Withdraw $50 from ATM4
.\transaction.ps1 -NodeId 4 -Type withdrawal -Amount 50
```

#### Monitor Token Circulation:

```
# Check status of all nodes
.\check-status.ps1

# Watch token circulation (check logs in each terminal)
# Token should appear at each node every ~20 seconds (4 nodes x 5 seconds)
```

## System Demonstration

---

The system correctly implements the Token Ring example from the project specification:

**Initial State:** Balance = \$1000, ATM1 creates and holds initial token

#### Example Scenario:

```
# Queue transactions
.\transaction.ps1 -NodeId 2 -Type withdrawal -Amount 200
.\transaction.ps1 -NodeId 3 -Type deposit -Amount 100
.\transaction.ps1 -NodeId 4 -Type withdrawal -Amount 500
```

#### Execution Sequence:

1. **ATM1:** No transaction → forwards token (Balance: \$1000)
2. **ATM2:** Receives token → executes withdrawal \$200 → forwards (Balance: \$800)

3. **ATM3**: Receives token → executes deposit \$100 → forwards (Balance: \$900)
4. **ATM4**: Receives token → executes withdrawal \$500 → forwards (Balance: \$400)
5. **ATM1**: Receives token with updated balance \$400

## Key Properties Demonstrated

- ☒ **Mutual Exclusion**: Only one node executes transactions at a time
- ☒ **No Shared Memory**: Balance travels in token, no global variables
- ☒ **Message Passing**: All communication via HTTP POST
- ☒ **Atomicity**: Each transaction is indivisible
- ☒ **Ordering**: Transactions execute in token arrival order
- ☒ **Audit Trail**: Complete logging of all operations

## Testing Scenarios

---

### 1. Normal Operation Test

```
.\start-nodes.ps1
Start-Sleep -Seconds 15
.\test-circulation.ps1
```

Expected: Token circulates through all 4 nodes, visible every 5 seconds

### 2. Transaction Test

```
.\transaction.ps1 -NodeId 1 -Type deposit -Amount 100
.\check-status.ps1
```

Expected: Balance increases by \$100 when ATM1 receives token

### 3. Multiple Transactions Test

```
.\transaction.ps1 -NodeId 2 -Type deposit -Amount 100
.\transaction.ps1 -NodeId 2 -Type deposit -Amount 200
.\transaction.ps1 -NodeId 2 -Type deposit -Amount 300
```

Expected: ATM2 executes one transaction per token reception

### 4. Coordinator Election Test

```
# Stop ATM4 (initial coordinator) manually (Ctrl+C in its terminal)
# Watch other nodes' logs - should see election messages
# Verify ATM3 becomes new coordinator
```

Expected: Bully Algorithm elects ATM3 as new coordinator

### 5. Token Regeneration Test (Scenario A)

```
# Step 1: Start system and do a transaction to change balance
.\start-nodes.ps1
Start-Sleep -Seconds 20
.\transaction.ps1 -NodeId 2 -Type deposit -Amount 500
Start-Sleep -Seconds 10 # Wait for transaction execution

# Step 2: Verify balance is $1500
.\check-status.ps1

# Step 3: Identify which node has token (check logs - look for "Token received")
# Example: Suppose ATM3 has the token

# Step 4: Stop the node holding the token (Ctrl+C in that terminal)
# For example, stop ATM3

# Step 5: Wait ~30 seconds - observe logs in other nodes:
# - "*** TOKEN LOST DETECTED ***" (after 30 second timeout)
# - Election messages: "Starting election (Bully Algorithm)"
# - New coordinator elected (e.g., ATM4)
# - "*** REGENERATING TOKEN *** (balance: $1500)"
# - "Token regenerated (ID: xxx, Balance: $1500)"

# Step 6: Verify circulation resumes with correct balance
.\check-status.ps1
# Balance should be $1500 (preserved from before crash)

# Step 7: Test ATM1 recovery behavior
# If ATM1 was stopped, restart it:
$env:NODE_ID=1; $env:PORT=3001; npm run start:dev
# Watch logs - should see: "Detected other active nodes - treating as recovery"
# Should NOT see: "Creating initial token" (prevents duplicate tokens!)
```

Expected:

- Token lost detection after 30 seconds
- Coordinator regenerates token with preserved balance (\$1500)
- Circulation resumes through active nodes
- ATM1 does not create duplicate token on recovery

## 6. Ring Reconstruction Test (Scenario B)

```
# Stop ATM3 manually (Ctrl+C in its terminal)
# Watch logs - should see ring reconstruction
# Verify ATM2 now forwards to ATM4 (skipping ATM3)
```

Expected: Coordinator reconstructs ring, circulation continues without failed node

## 7. Node Recovery Test (Scenario C)

```
# Stop ATM2 manually (Ctrl+C)
# Wait for ring reconstruction
# Restart ATM2
# Watch logs for recovery announcements
```

Expected: ATM2 reintegrates into ring, topology updates, circulation includes ATM2 again

## Logging

Each node produces structured logs:

- **[INFO]** Token received/forwarded events
- **[INFO]** Transaction execution start/completion
- **[INFO]** Token circulation monitoring (Scenario A detection)
- **[INFO]** Election messages (ELECTION, OK, COORDINATOR)
- **[INFO]** Coordinator activities (token regeneration, ring reconstruction, topology broadcast)
- **[INFO]** Node status changes (active, failed, recovered)
- **[SUCCESS]** Successful transaction with balance update

- **[ERROR]** Failed operations (insufficient funds, network errors, election failures)

Example logs for different scenarios:

**Normal Operation:**

```
[2026-02-07T14:47:25.229Z] [ATM1] [INFO] Token received
[2026-02-07T14:47:25.229Z] [ATM1] [INFO] No pending transactions. Holding token for 5000ms (display only)...
[2026-02-07T14:47:30.240Z] [ATM1] [INFO] Token forwarded to ATM2
```

**Transaction Execution:**

```
[2026-02-07T14:46:15.060Z] [ATM2] [INFO] Transaction started: deposit $200
[2026-02-07T14:46:15.060Z] [ATM2] [SUCCESS] Transaction completed: deposit $200 | New balance: $1100
[2026-02-07T14:46:15.060Z] [ATM2] [INFO] Token forwarded to ATM3
```

**Scenario A - Token Lost Detection and Regeneration:**

```
[2026-02-07T15:15:30.456Z] [ATM2] [INFO] *** TOKEN LOST DETECTED *** (not seen for 30.2s)
[2026-02-07T15:15:30.456Z] [ATM2] [INFO] Handling token lost scenario - starting election to regenerate token
[2026-02-07T15:15:30.460Z] [ATM2] [INFO] Starting election (Bully Algorithm)
[2026-02-07T15:15:30.475Z] [ATM4] [INFO] Received ELECTION from ATM2
[2026-02-07T15:15:35.680Z] [ATM4] [INFO] *** BECAME COORDINATOR ***
[2026-02-07T15:15:35.685Z] [ATM4] [INFO] I am coordinator - regenerating lost token
[2026-02-07T15:15:36.120Z] [ATM4] [INFO] *** REGENERATING TOKEN *** (balance: $1500)
[2026-02-07T15:15:36.125Z] [ATM4] [INFO] Token regenerated (ID: abc-123..., Balance: $1500). Restarting circulation...
[2026-02-07T15:15:36.130Z] [ATM4] [INFO] Token forwarded to ATM1
```

**Scenario B - Ring Reconstruction:**

```
[2026-02-07T14:46:00.007Z] [ATM4] [INFO] *** NODE FAILURE *** Removing ATM1 from ring
[2026-02-07T14:46:00.007Z] [ATM4] [INFO] *** RECONSTRUCTING RING TOPOLOGY ***
[2026-02-07T14:46:00.007Z] [ATM4] [INFO] ATM2 -> ATM3
[2026-02-07T14:46:00.007Z] [ATM4] [INFO] ATM3 -> ATM4
[2026-02-07T14:46:00.007Z] [ATM4] [INFO] ATM4 -> ATM2
[2026-02-07T14:46:00.015Z] [ATM4] [INFO] Broadcasting topology to all active nodes
```

**ATM1 Recovery (No Duplicate Token):**

```
[2026-02-07T14:47:02.900Z] [ATM1] [INFO] Checking if other nodes are active...
[2026-02-07T14:47:02.912Z] [ATM1] [INFO] Found active node: ATM2
[2026-02-07T14:47:02.913Z] [ATM1] [INFO] Detected other active nodes - treating as recovery (not creating token)
[2026-02-07T14:47:07.935Z] [ATM1] [INFO] *** ANNOUNCING RECOVERY TO COORDINATOR ***
```

## Implementation Details

### Key Configuration Constants

The system uses the following configurable constants (src/common/constants.ts):

- **INITIAL\_BALANCE** : \$1000 - Starting balance for the shared account
- **NUMBER\_OF\_NODES** : 4 - Total ATM nodes in the ring
- **TOKEN\_DISPLAY\_DELAY** : 5000ms - Educational delay for visualizing token circulation (set to 0 for production)
- **TOKEN\_CIRCULATION\_TIMEOUT** : 30000ms (30s) - Timeout for detecting lost tokens (Scenario A)
- **MAX\_RETRY\_ATTEMPTS** : 5 - Maximum retry attempts before declaring node failed
- **RETRY\_DELAY** : 2000ms - Delay between retry attempts
- **NODE\_RESPONSE\_TIMEOUT** : 3000ms - Timeout for node responses
- **ELECTION\_TIMEOUT** : 5000ms - Timeout for election responses
- **COORDINATOR\_TIMEOUT** : 10000ms - Timeout for coordinator announcements

### Technologies Used

- **NestJS**: TypeScript framework for scalable server applications
- **Axios**: HTTP client for token forwarding
- **UUID**: Unique token identification
- **PowerShell**: Automation and testing scripts

## Future Enhancements

- 
- ☐ **Web interface:** Real-time monitoring dashboard for token circulation, elections, and transactions
  - ☐ **Transaction history:** Persistent storage and audit trail of all operations in a database
  - ☐ **Advanced failure scenarios:** Testing and handling of Byzantine failures
  - ☐ **Performance monitoring:** Metrics collection for transaction throughput and election latency
  - ☐ **Multi-account support:** Handle multiple bank accounts with separate tokens

## License

---

This project is [MIT licensed](#).